Proxied Authentication in Flowsource using AWS ALB as the Proxy with Microsoft Azure AD provider

Table of Contents

Prerequisites ***2***

Environment Configuration Steps ***2***

Configuring Azure OAuth App **2**

Creating and Configuring AWS ALB **2**

Creating and Configuring AWS ALB to serve traffic to Flowsource 2

Configuring Authentication in the Target group of the created AWS ALB 2

Details reg. the AWS ALB 3

Flowsource Application Source Code updates ***4***

app-config.yaml **4**

packages/app/src/App.tsx update **4**

packages/backend/src/plugins/auth.ts update **4**

Flowsource DB Table updates ***8***

k8s YAML configurations ***8***

flowsource-svc.yaml **8**

Troubleshooting ***9***

Reference Links ***10***

Conclusion ***10***

You can configure an Application Load Balancer to securely authenticate users as they access your applications. This enables you to offload the work of authenticating users to your load balancer so that your applications can focus on their business logic.

This document provides a guide on how to set up Single Sign-On (SSO) offloading using AWS ALB as the Proxy with Microsoft Azure Active Directory (AD) in a Kubernetes environment.

Prerequisites

- A Kubernetes cluster
- AWS ALB
- Microsoft Azure AD
- OAuth App in Microsoft Azure
- kubectl CLI

Environment Configuration Steps

The setup involves creating Application Load Balancer in AWS (pointing to the node port service in Kubernetes cluster), and configuring it to use Azure AD for authentication.

Configuring Azure OAuth App

Register a new Azure OAuth app with the below details. (

Homepage URL: FLOWSOURCE_BASE_URL

Redirect URI: FLOWSOURCE_BASE_URL/oauth2/idpresponse

(Note: FLOWSOURCE_BASE_URL should be updated with the actual value.)

After the app is created, you should be able to find the Application (client) ID , Application Client Secret and Directory (tenant) ID. You'll need these values in your AWS ALB configuration.

Creating and Configuring AWS ALB

Creating and Configuring AWS ALB to serve traffic to Flowsource

1. **Create an Application Load Balancer**

Create an Application Load Balancer manually to point to the application. The Load Balancer should be in the same VPC/Subnet as the Kubernetes cluster.

2. **Configure Listeners with Target group**

Configure the listeners of the Load Balancer to listen on the source port (HTTPS) and forward the traffic to the target port (HTTP) of the NodePort service using a Target group.

3. **Configure SSL/TLS Certificate**

Configure the SSL/TLS certificate for the Load Balancer. This certificate can be obtained from a Certificate Manager.

4. **Configure Health Check**

Configure the health check of the Load Balancer to check the TCP connection on the target port of the NodePort service.

5. **Configure Target Instances**

Configure the target instances of the Load Balancer to be the instances in the Kubernetes cluster node group(s).

6. **Configure Security Groups**

Configure the security group of the Load Balancer to allow inbound traffic on the source port from the IPs or security groups that need to access the application.

Also, update the security group of the cluster/node group to allow the newly created Load Balancer on the ports range of the NodePort service.

Configuring Authentication in the Target group of the created AWS ALB

(*This below steps are referred from the [ALB Entra App registration]() document in backstage.*)

In the AWS console, configure ALB Authentication:

- Edit the ALB rule used to forward the traffic to Backstage and add a new Authenticate action. The action will have higher priority compared to the existing Forward to.
- Select OIDC under Authenticate
- Set Issuer to https://login.microsoftonline.com/{TENANT_ID}/v2.0, replacing {TENANT_ID} with the value of Directory (tenant) ID of the App Registration.
- Set Authorization endpoint to https://login.microsoftonline.com/{TENANT_ID}/oauth2/v2.0/authorize, replacing {TENANT_ID} with the value of Directory (tenant) ID of the App Registration.
- Set Token endpoint to https://login.microsoftonline.com/{TENANT_ID}/oauth2/v2.0/token, replacing {TENANT_ID} with the value of Directory (tenant) ID of the App Registration.
- Set User info endpoint to https://graph.microsoft.com/oidc/userinfo
- Set Client ID to the App Registration Application (client) Id
- Set Client secret to the App Registration client secret

Use the following advanced settings:

- Session cookie name = AWSELBAuthSessionCookie
- Set Session timeout as 15 minutes
    - Note: Ensure that this value is less than the access token lifetime configured in Azure AD.
- Scope = openid profile offline_access
    - The scopes openid and profile should be sufficient.
- Action on unauthenticated request = Authenticate (client reattempt)

Once you've saved the action, you should see an authentication flow be triggered against Entra ID when visiting the application URL with https.

Details reg. the AWS ALB

(*This below content is referred from* [User Claims Encoding and signature verification]() *document.*)

After your load balancer authenticates a user successfully, it sends the user claims received from the IdP to the target. The load balancer signs the user claim so that applications can verify the signature and verify that the claims were sent by the load balancer.

The ALB passes these two tokens

- x-amzn-oidc-data - The user claims, in JSON web tokens (JWT) format.
- x-amzn-oidc-accesstoken - The access token from the token endpoint, in plain text.
- x-amzn-oidc-identity - The subject field (sub) from the user info endpoint, in plain text.
    - Note: The sub claim is the best way to identify a given user.

Access tokens and user claims are different from ID tokens. Access tokens and user claims only allow access to server resources, while ID tokens carry additional information to authenticate a user. The Application Load Balancer creates a new access token when authenticating a user and only passes the access tokens and claims to the backend, however it does not pass the ID token information.

Flowsource Application Source Code updates

app-config.yaml

```
auth:
  environment: development
  providers:
    awsalb:
      issuer: https://login.microsoftonline.com/${AZURE_TENANT_ID}/v2.0
      region: ${AWS_ALB_REGION}
```

(Make sure that the environment variables referred here are set in the appropriate configmap)

packages/app/src/App.tsx update

```
import { ProxiedSignInPage } from '@backstage/core-components';

const app = createApp({
  apis,
  components: {
    SignInPage: props => {
      return (
        <ProxiedSignInPage {...props} provider="awsalb" />
      );
    },
  },

  // …

});
```

(Make sure that “awsalb” is set as the provider)

packages/backend/src/plugins/auth.ts update

```
import {
  createRouter,
  providers,
  defaultAuthProviderFactories,
} from '@backstage/plugin-auth-backend';
import { Router } from 'express';
import { PluginEnvironment } from '../types';
import { getUpdatedUserRefs, getUserRoles } from './helper/auth-helper';
import { initDatabase } from './database/initDatabase.service';
import { RoleMappingDatabaseService } from
'./database/roleMappingDatabase.service';
import { Knex } from 'knex';
import { DEFAULT_NAMESPACE, stringifyEntityRef, } from '@backstage/catalog-model';
// Added for Azure AD Auth
```

```
import { decodeJwt } from 'jose';
import { EmailToRoleMappingDatabaseService } from
'./database/emailToRoleMappingDatabase.service';

export default async function createPlugin(
  env: PluginEnvironment,
): Promise<Router> {

  // Initialize/create the database
  await initDatabase({
    logger: env.logger,
    database: env.database,
  });

  // Get database client
  const db: Knex = await env.database.getClient();
  const roleMappingDatabaseService = new RoleMappingDatabaseService(db);
  const emailToRoleMappingDatabaseService = new
EmailToRoleMappingDatabaseService(db);

  // Create the router
  return await createRouter({
    logger: env.logger,
    config: env.config,
    database: env.database,
    discovery: env.discovery,
    tokenManager: env.tokenManager,
    providerFactories: {
      ...defaultAuthProviderFactories,
      awsalb: providers.awsAlb.create({
        authHandler: async (obj) => {
          const fullProfile = obj?.fullProfile;
          const accessToken = obj?.accessToken;
          let email: string | undefined = undefined;
          if (fullProfile.emails && fullProfile.emails.length > 0) {
            const [firstEmail] = fullProfile.emails;
            email = firstEmail.value;
          }

          const displayName: string | undefined =
            fullProfile.displayName ?? fullProfile.username ?? fullProfile.id;

          return {
            profile: {
              email,
              displayName,
              accessToken,
            },
          };
        },
        signIn: {
```

```
          resolver: async (obj, ctx) => {
            const profile: any = obj?.profile;
            if (!profile.email) {
              throw new Error('Profile contained no email');
            }

            const [id] = profile.email.split('@');
            if (!id) {
              throw new Error('Invalid email format');
            }

            const userRef = stringifyEntityRef({
              kind: 'User',
              name: profile.email,
              namespace: DEFAULT_NAMESPACE,
            });

            let roleArray: any = [];
            if (profile.accessToken) {
              const accessTokenDecoded: any = decodeJwt(profile.accessToken);
              if(accessTokenDecoded.roles) {
                roleArray = accessTokenDecoded.roles;
              } else {
                console.warn("roles not found in accessToken");
              }
            }

            if (!roleArray || roleArray.length === 0) {
              // Get the authProvider roles from the emailToRoleMapping database
service for the given email
              roleArray = await getUserRoles(profile.email,
emailToRoleMappingDatabaseService);
            }

            if (!roleArray || roleArray.length === 0) {
              // If no roles are found for the user, throw an error
              console.error('Unauthorized - No roles found for the user');
              throw new Error('Unauthorized - No roles found for the user');
            }

            const userRefs = await getUpdatedUserRefs(roleArray, 'microsoft',
roleMappingDatabaseService);

            const { token } = await ctx.issueToken({
              claims: {
                sub: userRef,
                ent: userRefs,
              },
            });

            return { id, token };
```

```
          },
        },
      }),
    },
  });
}
```

(Note: Here, the role information of the user are got from the database table, by calling the method getUserRoles. This logic will be used when the access token does not contain any role information).

Flowsource DB Table updates

Include the Email to Auth Provider role mapping in the DB “backstage_plugin_auth” in the table “email_to_role_mappings”, for the required admin users.

E.g.:

insert into email_to_role_mappings values('arutsudar.a@cognizant.com','catalog-admin');

k8s YAML configurations

flowsource-svc.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: flowsource-service
spec:
  ports:
  - port: 7007
    protocol: TCP
    targetPort: 7007
  selector:
    app: flowsource
  sessionAffinity: None
  type: NodePort
```

(Make sure to use the Node port value from this service in the Load Balancer configurations)

Troubleshooting

1) **Error Message**: "Needs permission to access resources in your organization that only an admin can grant. Please ask an admin to grant permission to this app before you can use it."

   Cause: This issue is related to the Azure AD configurations.

   Recommended Actions:
   1) Contact Azure AD team
      We recommend checking with your “Azure AD“ team to resolve this issue.
   2) Enterprise Application Settings
      In the Azure portal, in Enterprise Application’s service, look for the User Settings section. Check the configurations related to these 2 points, and update them as necessary:
      i) Users can consent to apps accessing company data on their behalf.
      ii) Users can request admin consent to apps they are unable to consent to.

   Additional Resources:
   - Check this post in the official Microsoft site.
   - Refer to this post from Stackoverflow for more details.

Reference Links

- https://github.com/backstage/backstage/blob/master/contrib/docs/tutorials/aws-alb-aad-oidc-auth.md
- https://docs.aws.amazon.com/elasticloadbalancing/latest/application/listener-authenticate-users.html

Conclusion

After following these steps, your application should be protected by the AWS ALB as the Proxy, which uses Azure AD for authentication. Users will be redirected to Microsoft login page for sign-in and then redirected back to your application.